School of Computing

Department of Computer Science and Engineering

USE CASE — REAL LIFE EXAMPLE

SMART PARKING MANAGEMENT SYSTEM

Program – Department Name	B.Tech – CSE
Academic Year / Semester	2025–26
Faculty Name	Dr. S. Suchitra
Course Code / Course Name	10211CS228 / Problem Solving and Testing Using Java
Slot No. & Date	S8-2L1L5
Student Name	M. Chanikya
Title of the Complex Engineering Problem	Smart Parking Management System
Engineering Problem / Problems Description / Complexity Addressed	Design and develop a console-based application to manage parking operations in a multi-level parking area. The system allocates available slots, records vehicle details, manages entry and exit, calculates fees, supports VIP and emergency priority, and generates parking analytics.
CO's Mapped	CO1, CO2, CO3, CO4, CO5 (confirm with course faculty if a prescribed mapping is required)
PO's / PSO's Mapped	PO1, PO2, PO3, PO5, PSO2 (confirm with course faculty if a prescribed mapping is required)
SDG Mapped with Number	SDG 9 – Industry, Innovation and Infrastructure
Justification for SDG Mapped	The system applies software-based automation to improve the efficiency of parking infrastructure, reduce manual allocation effort, and support better use of available parking resources.

Proof for the solved Complex Engineering Problem: Source code and sample console output are included in this document.

TABLE OF CONTENTS

S. No.	Contents	Page No.
1	Case Study / Use Case	3
2	Problem Statement	4
3	Java Source Code	5–7
4	Sample Output Screenshots (console-style output)	8
5	Result	9

Document structure follows the supplied sample: case study, problem statement, code, output, and result.

1. CASE STUDY / USE CASE

Smart Parking Management System

The Smart Parking Management System is a console-based Java application intended to coordinate vehicle parking in a multi-level facility. It keeps track of available and occupied slots, records vehicle entry time, processes vehicle exit, and calculates parking charges based on the duration of stay. The design also demonstrates priority handling for VIP and emergency vehicles through a priority queue.

Purpose

The purpose is to reduce manual parking allocation, avoid assigning an already occupied slot, improve visibility of parking availability, and produce basic operational reports for the parking facility.

Main Actors

Driver / Vehicle Owner: presents a vehicle for entry or exit. **Parking Attendant:** uses the console menu to register vehicles and complete parking operations. **System Administrator:** reviews occupancy and peak-hour summaries.

Primary Use Cases

Use case	Description
Register vehicle entry	Capture registration number, vehicle type and priority; allocate an available slot.
Allocate parking slot	Choose the nearest available slot and assign its parking level.
Process vehicle exit	Find the parked vehicle, calculate duration and fee, release its slot.
Manage priority	Queue VIP and emergency arrivals by priority before assigning slots.
View parking analytics	Show occupied/free capacity by level and identify the most occupied level.
Generate report	Display current parked vehicles and total occupancy information.

Assumptions: this academic version uses in-memory data and a fixed hourly fee. Data is cleared when the program exits. For a production system, persistent storage, authentication, payment integration, and real sensor data would be required.

2. PROBLEM STATEMENT

Design and develop a console-based Smart Parking Management System that efficiently manages vehicle parking operations in a multi-level parking area. Manual parking management can lead to delays, inaccurate slot tracking, inefficient use of levels, and inconsistent fee calculation. The proposed application automates slot allocation and maintains a live record of parked vehicles.

Functional Requirements

Vehicle Entry Management

• Allocate the nearest available slot.
• Store the registration number, vehicle type, priority category, entry timestamp, and assigned slot.
• Automatically identify the parking level from the allocated slot.

Vehicle Exit Management

• Locate a parked vehicle by registration number.
• Calculate the parking duration from entry to exit.
• Calculate the parking fee and release the occupied slot.

Parking Analytics

• Report free and occupied slots for each level.
• Identify the most occupied level.
• Record entry counts by hour to estimate peak parking hours.

Priority Parking

• Support VIP and emergency vehicles.
• Use a priority queue to process higher-priority arrivals first. Emergency vehicles are ranked above VIP vehicles; standard vehicles are handled after priority arrivals.

Data Structures and Java Concepts

The application uses classes to represent vehicles and parking slots, a list to maintain parked vehicles, a priority queue for priority arrivals, date-time utilities for duration calculations, and methods to separate entry, exit, fee calculation, and reporting logic. Input validation prevents duplicate registration numbers and invalid menu selections.

Fee policy used in this example: ■30 per started hour. The fee is calculated using the ceiling of elapsed minutes divided by 60, with a minimum charge of one hour. This is a sample policy and can be changed to match actual parking rules.

3. JAVA SOURCE CODE

Save the following source as **SmartParkingManagementSystem.java**. Compile with a Java compiler and run from the console. This example uses three levels with four slots per level.

```
import java.time.Duration;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.*;

/**
 * Smart Parking Management System
 * Console-based educational example.
 */
public class SmartParkingManagementSystem {
    static final int LEVELS = 3;
    static final int SLOTS_PER_LEVEL = 4;
    static final double RATE_PER_HOUR = 30.0;
    static final DateTimeFormatter TIME_FMT =
            DateTimeFormatter.ofPattern("dd-MM-yyyy HH:mm:ss");

    static class Vehicle {
        String number, type;
        int priority; // 2 = emergency, 1 = VIP, 0 = standard
        LocalDateTime entryTime;

        Vehicle(String number, String type, int priority) {
            this.number = number.toUpperCase(Locale.ROOT);
            this.type = type;
            this.priority = priority;
            this.entryTime = LocalDateTime.now();
        }
    }

    static class ParkingSlot {
        int level, slotNumber;
        Vehicle vehicle;

        ParkingSlot(int level, int slotNumber) {
            this.level = level;
            this.slotNumber = slotNumber;
        }

        boolean isFree() { return vehicle == null; }
    }

    static final List<ParkingSlot> slots = new ArrayList<>();
    static final Map<String, ParkingSlot> parked = new LinkedHashMap<>();
    static final PriorityQueue<Vehicle> priorityQueue =
            new PriorityQueue<>((a, b) -> {
                int byPriority = Integer.compare(b.priority, a.priority);
                if (byPriority != 0) return byPriority;
                return a.entryTime.compareTo(b.entryTime);
            });
    static final Map<Integer, Integer> entriesByHour = new TreeMap<>();
    static final Scanner input = new Scanner(System.in);

    public static void main(String[] args) {
        for (int level = 1; level <= LEVELS; level++)
            for (int slot = 1; slot <= SLOTS_PER_LEVEL; slot++)
                slots.add(new ParkingSlot(level, slot));

        while (true) {
            System.out.println("\\n=== SMART PARKING MANAGEMENT SYSTEM ===");
            System.out.println("1. Vehicle Entry");
            System.out.println("2. Vehicle Exit");
            System.out.println("3. Parking Availability");
            System.out.println("4. Parking Report / Analytics");
            System.out.println("5. Process Priority Queue");
            System.out.println("0. Exit");
            System.out.print("Choose option: ");
            int choice = readInt();

            switch (choice) {
                case 1: registerArrival(); break;
                case 2: vehicleExit(); break;
                case 3: showAvailability(); break;
                case 4: showReport(); break;
                case 5: processPriorityQueue(); break;
                case 0:
                    System.out.println("System closed. Thank you.");
```

3. JAVA SOURCE CODE (CONTINUED)

```
                return;
            default:
                System.out.println("Invalid option. Try again.");
        }
    }
}

static int readInt() {
    try { return Integer.parseInt(input.nextLine().trim()); }
    catch (NumberFormatException ex) { return -1; }
}

static void registerArrival() {
    System.out.print("Registration number: ");
    String number = input.nextLine().trim().toUpperCase(Locale.ROOT);
    if (number.isEmpty() || parked.containsKey(number)) {
        System.out.println("Invalid or already parked registration number.");
        return;
    }

    System.out.print("Vehicle type (Car/Bike/Van): ");
    String type = input.nextLine().trim();
    System.out.print("Priority (0=Standard, 1=VIP, 2=Emergency): ");
    int priority = readInt();
    if (type.isEmpty() || priority < 0 || priority > 2) {
        System.out.println("Invalid vehicle type or priority.");
        return;
    }

    Vehicle vehicle = new Vehicle(number, type, priority);
    if (priority > 0) {
        priorityQueue.offer(vehicle);
        System.out.println("Priority vehicle queued. Select option 5 to allocate in priority order.");
    } else {
        allocate(vehicle);
    }
}

static void processPriorityQueue() {
    if (priorityQueue.isEmpty()) {
        System.out.println("No priority vehicles are waiting.");
        return;
    }
    while (!priorityQueue.isEmpty()) {
        Vehicle vehicle = priorityQueue.poll();
        allocate(vehicle);
    }
}

static void allocate(Vehicle vehicle) {
    for (ParkingSlot slot : slots) {
        if (slot.isFree()) {
            slot.vehicle = vehicle;
            parked.put(vehicle.number, slot);
            entriesByHour.merge(vehicle.entryTime.getHour(), 1, Integer::sum);
            System.out.printf("Allocated Level %d, Slot %d to %s (%s).%n",
                    slot.level, slot.slotNumber, vehicle.number,
                    priorityName(vehicle.priority));
            System.out.println("Entry time: " + vehicle.entryTime.format(TIME_FMT));
            return;
        }
    }
    System.out.println("Parking full. Could not allocate slot to " + vehicle.number);
}

static String priorityName(int priority) {
    if (priority == 2) return "Emergency";
    if (priority == 1) return "VIP";
    return "Standard";
}

static void vehicleExit() {
    System.out.print("Registration number to exit: ");
    String number = input.nextLine().trim().toUpperCase(Locale.ROOT);
    ParkingSlot slot = parked.remove(number);
    if (slot == null) {
        System.out.println("Vehicle not found in the parking area.");
        return;
    }

    LocalDateTime exitTime = LocalDateTime.now();
    long minutes = Math.max(1, Duration.between(
            slot.vehicle.entryTime, exitTime).toMinutes());
    long hours = Math.max(1, (minutes + 59) / 60);
    double fee = hours * RATE_PER_HOUR;
```

```
        System.out.println("Vehicle: " + number);
        System.out.println("Entry: " + slot.vehicle.entryTime.format(TIME_FMT));
        System.out.println("Exit : " + exitTime.format(TIME_FMT));
        System.out.println("Duration: " + minutes + " minute(s)");
        System.out.printf("Parking fee: ■%.2f (%d billed hour(s))%n", fee, hours);
        System.out.printf("Released Level %d, Slot %d.%n", slot.level, slot.slotNumber);
        slot.vehicle = null;
    }

    static void showAvailability() {
        for (int level = 1; level <= LEVELS; level++) {
            int occupied = 0;
            for (ParkingSlot slot : slots)
                if (slot.level == level && !slot.isFree()) occupied++;
            System.out.printf("Level %d: %d occupied, %d available%n",
                    level, occupied, SLOTS_PER_LEVEL - occupied);
        }
    }

    static void showReport() {
        System.out.println("\\n--- PARKING REPORT ---");
        System.out.printf("Total slots: %d | Occupied: %d | Available: %d%n",
                slots.size(), parked.size(), slots.size() - parked.size());
        showAvailability();

        int bestLevel = 1, highest = -1;
        for (int level = 1; level <= LEVELS; level++) {
            int occupied = 0;
            for (ParkingSlot slot : slots)
                if (slot.level == level && !slot.isFree()) occupied++;
            if (occupied > highest) {
                highest = occupied;
                bestLevel = level;
            }
        }
        System.out.printf("Most occupied level: Level %d (%d/%d slots)%n",
                bestLevel, highest, SLOTS_PER_LEVEL);

        if (entriesByHour.isEmpty()) {
            System.out.println("Peak entry hour: no entries recorded yet.");
        } else {
            int peakHour = -1, peakCount = -1;
            for (Map.Entry<Integer, Integer> e : entriesByHour.entrySet()) {
                if (e.getValue() > peakCount) {
                    peakHour = e.getKey();
                    peakCount = e.getValue();
                }
            }
            System.out.printf("Peak entry hour (recorded entries): %02d:00 (%d vehicle(s))%n",
                    peakHour, peakCount);
        }

        if (parked.isEmpty()) System.out.println("No vehicles currently parked.");
        else {
            System.out.println("Currently parked vehicles:");
            for (Map.Entry<String, ParkingSlot> e : parked.entrySet()) {
                ParkingSlot s = e.getValue();
                System.out.printf("%s | Level %d | Slot %d | %s%n",
                        e.getKey(), s.level, s.slotNumber,
                        priorityName(s.vehicle.priority));
            }
        }
    }
}
```

4. OUTPUT SCREENSHOT / SAMPLE CONSOLE OUTPUT

The following console-style sample illustrates the expected interaction. Actual timestamps and slot occupancy will depend on when the program is run.

```
=== SMART PARKING MANAGEMENT SYSTEM ===
1. Vehicle Entry
2. Vehicle Exit
3. Parking Availability
4. Parking Report / Analytics
5. Process Priority Queue
0. Exit
Choose option: 1
Registration number: TN01AB1234
Vehicle type (Car/Bike/Van): Car
Priority (0=Standard, 1=VIP, 2=Emergency): 0
Allocated Level 1, Slot 1 to TN01AB1234 (Standard).
Entry time: 26-10-2026 09:15:00

Choose option: 1
Registration number: TN09EM7777
Vehicle type (Car/Bike/Van): Van
Priority (0=Standard, 1=VIP, 2=Emergency): 2
Priority vehicle queued. Select option 5 to allocate in priority order.

Choose option: 5
Allocated Level 1, Slot 2 to TN09EM7777 (Emergency).
Entry time: 26-10-2026 09:16:00

Choose option: 3
Level 1: 2 occupied, 2 available
Level 2: 0 occupied, 4 available
Level 3: 0 occupied, 4 available

Choose option: 4
--- PARKING REPORT ---
Total slots: 12 | Occupied: 2 | Available: 10
Level 1: 2 occupied, 2 available
Level 2: 0 occupied, 4 available
Level 3: 0 occupied, 4 available
Most occupied level: Level 1 (2/4 slots)
Peak entry hour (recorded entries): 09:00 (2 vehicle(s))
Currently parked vehicles:
TN01AB1234 | Level 1 | Slot 1 | Standard
TN09EM7777 | Level 1 | Slot 2 | Emergency

Choose option: 2
Registration number to exit: TN01AB1234
Vehicle: TN01AB1234
Duration: 5 minute(s)
Parking fee: ■30.00 (1 billed hour(s))
Released Level 1, Slot 1.
```

Note: This is a formatted sample output, not a captured screenshot from an actual execution. Run the Java program to capture authentic output for submission if required.

5. RESULT

The Smart Parking Management System design and Java implementation demonstrate the core operations of a multi-level parking facility. The program supports vehicle registration, slot allocation, priority queuing for VIP and emergency vehicles, exit processing, duration-based fee calculation, availability checks, and basic parking analytics.

The intended objective of demonstrating structured problem solving and Java programming concepts is achieved in this educational implementation. The program can be extended with a database, configurable rates, separate emergency-only slots, real-time sensors, a more detailed peak-hour history, and automated report export.

Testing considerations

1. Register a standard vehicle and verify that a free slot is assigned.
2. Register a VIP and an emergency vehicle, then process the queue and verify emergency-first order.
3. Attempt to register a duplicate vehicle number and verify rejection.
4. Attempt to exit an unknown vehicle and verify the error message.
5. Fill all slots and verify that additional arrivals are reported as unable to park.
6. Exit a parked vehicle and verify that the slot becomes available and the fee is calculated.

Important implementation note

This is a console-based academic example. Priority arrivals are placed in a priority queue and are allocated when option 5 is selected; standard arrivals are allocated immediately. The sample tracks peak entry hour from in-memory entry counts for the current run, rather than maintaining a long-term historical database.